Extending Your A*/ANA* MSA Implementation to Use PAM250 Costs

CSE Research — Implementation Notes

March 15, 2026

Contents

1 What Changes and What Stays the Same **1**

2 Background: Why PAM250? **2**
- 2.1 The problem with unit costs . . . 2
- 2.2 What PAM250 is . . . 2
- 2.3 The sign issue — and why it does not affect your code . . . 2

3 Step 1: Sequence Encoding **3**

4 Step 2: The PAM250 Cost Table **3**

5 Step 3: Edge Cost Computation **4**

6 Step 4: Pairwise Suffix DP (One-Line Change) **5**

7 Step 5: Putting It All Together **6**

8 Testing and Verification **6**
- 8.1 Start with two sequences (pairwise alignment) . . . 6
- 8.2 Verify f-monotonicity . . . 7
- 8.3 Expected cost ranges . . . 7

9 Benchmark Sequences **7**

10 Reference **8**

Purpose of this guide. You already have a working A* and ANA* implementation for multiple sequence alignment (MSA) using unit costs (longest common subsequence). This guide explains how to extend it to use the PAM250 substitution cost function, which is biologically meaningful and has non-unit integer costs. The changes to your existing code are small — mostly a cost table swap — and the overall search logic stays exactly the same.

1 What Changes and What Stays the Same

Before diving in, it helps to be clear about exactly how much work is involved.

What stays exactly the same

- The K-dimensional DAG search structure
- The A* and ANA* search loops
- The priority queue and open/closed list management
- Goal detection and path reconstruction
- The *structure* of the SOP heuristic and suffix DP

What changes

- **The cost table** (swap unit costs for PAM250 values)
- **One line** in the pairwise suffix DP inner loop
- **Sequence encoding** (characters instead of integers, or a simple mapping from characters to indices)

That is genuinely all. The Johnson reweighting that makes A* work correctly with mixed-sign costs is *already handled implicitly* by your existing A* f-value computation — more on this in Section 2.3.

2 Background: Why PAM250?

2.1 The problem with unit costs

In the LCS formulation, every substitution or gap costs 1. This means all edge costs in the alignment DAG are equal, and bucket aggregation provides little advantage over a single-level structure.

For the bucket queue paper, we need a cost function where edge costs *vary significantly*, so that aggregation over buckets is meaningful. PAM250 provides this: after negation for minimization, costs range from 0 (identical or highly conserved residue pairs) up to about 15 (radical substitutions), with most values spread across that range.

2.2 What PAM250 is

PAM250 (Point Accepted Mutation, 250 evolutionary units) is a substitution scoring matrix derived from an evolutionary model of protein sequences. The entry $M[a][b]$ gives a log-odds score for substituting amino acid a with amino acid b:

- **Positive** entries: conservative substitutions that occur more often than expected by chance (e.g., Ile→Val, Leu→Met).
- **Zero** entries: substitutions at roughly chance frequency.
- **Negative** entries: radical substitutions that are unlikely (e.g., Cys→Trp).

Because we are *minimizing* alignment cost rather than maximizing score, we negate the matrix: $\text{COST}[a][b] = -M[a][b]$. After negation, conservative substitutions are cheap and radical ones are expensive.

2.3 The sign issue — and why it does not affect your code

After negation, the PAM250 cost table contains *negative* values for identical and highly conservative residue pairs (e.g., COST[W][W] $= -17$). Negative edge costs would normally break Dijkstra's algorithm. However:

> **Key insight.** A* with a *consistent* heuristic automatically performs a transformation (called Johnson reweighting) that makes all effective edge costs non-negative. This transformation happens implicitly through the f-value computation $f(n) = g(n) + h(n)$, which your code already computes. You do not need to implement anything extra.
>
> The SOP heuristic is consistent under PAM250 costs. This was proved by Lermen and Reinert (2000), who used exactly this approach in their GSA program.

To see why this works: when A* computes the priority of a neighbor r of the current node n, it uses

$$f(r) = g(n) + c(n, r) + h(r).$$

This is equivalent to using the *reweighted* edge cost $c'(n, r) = c(n, r) + h(r) - h(n)$, which is non-negative whenever h is consistent. Since your code already computes $f(r)$ this way, no additional changes are needed to handle the sign issue.

As a sanity check during development, you can add the assertion:

```
// f-value should never decrease when h is consistent
assert(f_neighbor >= f_current);
```

If this assertion ever fires, something is wrong with the heuristic computation.

3 Step 1: Sequence Encoding

Your LCS implementation likely represents each sequence as an array of integers (e.g., 0 and 1 for a binary alphabet, or character codes). PAM250 is defined over the 20 standard amino acids plus a gap character. The simplest approach is a lookup table from characters to indices.

```
// Standard amino acid ordering used by PAM250 (NCBI convention)
static const char AA_ORDER[] = "ARNDCQEGHILKMFPSTWYV";
static const int  NUM_AA     = 20;
static const int  GAP_IDX    = 20;   // index for gap character '-'

// Convert a character to its index (0..19 for amino acids, 20 for gap)
int aa_to_idx(char c) {
    if (c == '-') return GAP_IDX;
    c = toupper(c);
    for (int i = 0; i < NUM_AA; ++i)
        if (AA_ORDER[i] == c) return i;
    return -1;  // should not happen with valid input
}

// Call this once to encode an input sequence
std::vector<int> encode_sequence(const std::string& s) {
    std::vector<int> enc(s.size());
    for (int i = 0; i < (int)s.size(); ++i)
```

```
        enc[i] = aa_to_idx(s[i]);
    return enc;
}
```

Listing 1: Alphabet encoding — add to your existing code

4 Step 2: The PAM250 Cost Table

Replace your unit-cost table with the negated PAM250 matrix. This is the most mechanical part of the extension — just copy the table below into your code and call `init_cost_table` at startup.

```
// Global cost table: COST[a][b] = -PAM250[a][b]
// Indices 0..19 for amino acids (order: ARNDCQEGHILKMFPSTWYV)
// Index 20 for gap character '-'
int COST[21][21];

void init_cost_table(int gap_penalty = 8) {
    // Negated PAM250 matrix (20x20 amino acid block)
    static const int PAM250[20][20] = {
    /*        A   R   N   D   C   Q   E   G   H   I   L   K   M   F   P
       S   T   W   Y   V */
    /* A */ { 2,-2, 0, 0,-2, 0, 0, 1,-1,-1,-2,-1,-1,-3, 1, 1, 1,-6,-3, 0},
    /* R */ {-2, 6, 0,-1,-4, 1,-1,-3, 2,-2,-3, 3, 0,-4, 0, 0,-1, 2,-4,-2},
    /* N */ { 0, 0, 2, 2,-4, 1, 1, 0, 2,-2,-3, 1,-2,-3, 0, 1, 0,-4,-2,-2},
    /* D */ { 0,-1, 2, 4,-5, 2, 3, 1, 1,-2,-4, 0,-3,-6,-1, 0, 0,-7,-4,-2},
    /* C */ {-2,-4,-4,-5,12,-5,-5,-3,-3,-2,-6,-5,-5,-4,-3, 0,-2,-8, 0,-2},
    /* Q */ { 0, 1, 1, 2,-5, 4, 2,-1, 3,-2,-2, 1,-1,-5, 0,-1,-1,-5,-4,-2},
    /* E */ { 0,-1, 1, 3,-5, 2, 4, 0, 1,-2,-3, 0,-2,-5,-1, 0, 0,-7,-4,-2},
    /* G */ { 1,-3, 0, 1,-3,-1, 0, 5,-2,-3,-4,-2,-3,-5, 0, 1, 0,-7,-5,-1},
    /* H */ {-1, 2, 2, 1,-3, 3, 1,-2, 6,-2,-2, 0,-2,-2, 0,-1,-1,-3, 0,-2},
    /* I */ {-1,-2,-2,-2,-2,-2,-2,-3,-2, 5, 2,-2, 2, 1,-2,-1, 0,-5,-1, 4},
    /* L */ {-2,-3,-3,-4,-6,-2,-3,-4,-2, 2, 6,-3, 4, 2,-3,-3,-2,-2,-1, 2},
    /* K */ {-1, 3, 1, 0,-5, 1, 0,-2, 0,-2,-3, 5, 0,-5,-1, 0, 0,-3,-4,-2},
    /* M */ {-1, 0,-2,-3,-5,-1,-2,-3,-2, 2, 4, 0, 6, 0,-2,-2,-1,-4,-2, 2},
    /* F */ {-3,-4,-3,-6,-4,-5,-5,-5,-2, 1, 2,-5, 0, 9,-5,-3,-3, 0, 7,-1},
    /* P */ { 1, 0, 0,-1,-3, 0,-1, 0, 0,-2,-3,-1,-2,-5, 6, 1, 0,-6,-5,-1},
    /* S */ { 1, 0, 1, 0, 0,-1, 0, 1,-1,-1,-3, 0,-2,-3, 1, 2, 1,-2,-3,-1},
    /* T */ { 1,-1, 0, 0,-2,-1, 0, 0,-1, 0,-2, 0,-1,-3, 0, 1, 3,-5,-3, 0},
    /* W */ {-6, 2,-4,-7,-8,-5,-7,-7,-3,-5,-2,-3,-4, 0,-6,-2,-5,17, 0,-6},
    /* Y */ {-3,-4,-2,-4, 0,-4,-4,-5, 0,-1,-1,-4,-2, 7,-5,-3,-3, 0,10,-2},
    /* V */ { 0,-2,-2,-2,-2,-2,-2,-1,-2, 4, 2,-2, 2,-1,-1,-1, 0,-6,-2, 4}
    };

    // Fill the 20x20 amino acid block (negated)
    for (int a = 0; a < 20; ++a)
        for (int b = 0; b < 20; ++b)
            COST[a][b] = -PAM250[a][b];

    // Gap paired with any amino acid costs gap_penalty
    for (int a = 0; a < 20; ++a) {
        COST[a][GAP_IDX] = gap_penalty;
        COST[GAP_IDX][a] = gap_penalty;
    }
```

```

    // Gap paired with gap: does not occur in valid alignments
    COST[GAP_IDX][GAP_IDX] = 0;
}
```

Listing 2: PAM250 cost table — replaces your existing cost function

Note on signs. The raw COST table contains negative values for identical and conservative residue pairs (e.g., COST[W][W] = −17). This is expected. As explained in Section 2.3, these negative values are handled automatically by A*'s f-value computation. Do not add a constant offset to make all values non-negative — that would distort path lengths and break optimality.

5 Step 3: Edge Cost Computation

Your existing `edge_cost` function probably computes a unit cost for each mismatched or gapped pair. Replace it with the SOP cost using the new table. The structure of the function is identical — only the cost lookup changes.

```
// Compute the SOP edge cost for one alignment column.
// chars[i] = encoded residue (0..19) or GAP_IDX (20) for sequence i.
// K = number of sequences.
int edge_cost(const int* chars, int K) {
    int cost = 0;
    for (int i = 0; i < K; ++i)
        for (int j = i + 1; j < K; ++j)
            cost += COST[chars[i]][chars[j]];
    return cost;
}
```

Listing 3: SOP edge cost — replace your existing version

When generating neighbors of a node $(p_1, \ldots, p_K)$, build the `chars` array for each advance vector $\delta \in \{0,1\}^K \setminus \{\mathbf{0}\}$ as follows:

```
// For advance vector delta (bitmask over K sequences):
void build_chars(const int* pos,            // current position tuple
                 const int** seqs,          // encoded sequences
                 int delta,                 // bitmask: bit i set =
                     advance seq i
                 int K,
                 int* chars)                // output: character for each
                     sequence
{
    for (int i = 0; i < K; ++i) {
        if (delta & (1 << i))
            chars[i] = seqs[i][pos[i]];    // residue at current position
        else
            chars[i] = GAP_IDX;            // gap
    }
}
```

Listing 4: Building the column character array for a neighbor

6 Step 4: Pairwise Suffix DP (One-Line Change)

Your SOP heuristic precomputes a table $D_{ij}[p][q]$ = optimal alignment cost for the suffix of sequence i starting at position p and the suffix of sequence j starting at position q. The only change needed is to use `COST` instead of the unit cost in the inner loop.

Original inner loop (unit cost)

```
// Unit cost version -- what you currently have
int sub  = (seq_i[p] != seq_j[q] ? 1 : 0) + D[p+1][q+1];
int del  = 1 + D[p+1][q];
int ins  = 1 + D[p][q+1];
D[p][q]  = std::min({sub, del, ins});
```

Updated inner loop (PAM250)

```
// PAM250 version -- the only change needed in this function
int sub  = COST[seq_i[p]][seq_j[q]] + D[p+1][q+1];
int del  = COST[seq_i[p]][GAP_IDX]  + D[p+1][q];
int ins  = COST[GAP_IDX][seq_j[q]]  + D[p][q+1];
D[p][q]  = std::min({sub, del, ins});
```

The boundary conditions also use `COST` with `GAP_IDX`:

```
// Boundary: suffix of seq_i aligned entirely against gaps
for (int p = len_i - 1; p >= 0; --p)
    D[p][len_j] = (len_i - p) * COST[seq_i[p]][GAP_IDX];

// Boundary: suffix of seq_j aligned entirely against gaps
for (int q = len_j - 1; q >= 0; --q)
    D[len_i][q] = (len_j - q) * COST[GAP_IDX][seq_j[q]];

D[len_i][len_j] = 0;
```

7 Step 5: Putting It All Together

The changes described above are the complete extension. Here is the full sequence of steps to integrate them:

1. **Call `init_cost_table()` at program startup**, before any sequences are read or any DP is computed.
2. **Encode input sequences** using `encode_sequence` immediately after reading them. Store as `std::vector<int>` and pass integer pointers to all downstream functions.
3. **Rerun the pairwise suffix DP** for each pair (i, j) using the updated inner loop. This gives the SOP heuristic tables D_{ij}.
4. **Use the updated `edge_cost`** when expanding nodes in A* / ANA*. No other changes to the search loop.
5. **The heuristic lookup** (`sop_heuristic`) is unchanged — it still sums $D_{ij}[p_i][p_j]$ over all pairs, which now reflects PAM250 costs automatically.

Summary of changed functions.

Function	Change	Effort
`init_cost_table`	New function, copy from above	5 min
`aa_to_idx` / `encode_sequence`	New functions	10 min
`edge_cost`	Replace cost lookup	5 min
`precompute_pair` (suffix DP)	Change 3 lines in inner loop	10 min
A* / ANA* search loop	**No change**	—
Priority queue	**No change**	—
Goal detection / path reconstruction	**No change**	—

8 Testing and Verification

8.1 Start with two sequences (pairwise alignment)

Before testing the full K-sequence case, verify correctness on two sequences. With $K = 2$, the MSA reduces to standard pairwise alignment, and you can check against a reference implementation of Needleman-Wunsch with PAM250.

A simple Python script can serve as a reference:

```python
# Needleman-Wunsch with PAM250 and linear gap penalty
# Use BioPython: pip install biopython
from Bio.Align import PairwiseAligner
from Bio import Align

aligner = PairwiseAligner()
aligner.substitution_matrix = Align.substitution_matrices.load("PAM250")
aligner.open_gap_score    = -8   # match your gap_penalty
aligner.extend_gap_score  = -8   # linear gaps: open == extend

score = aligner.score("ACGT...", "ACGT...")
# Your A* result should equal -score (negated for minimization)
```

Listing 5: Reference pairwise alignment in Python

8.2 Verify f-monotonicity

Add a debug assertion to your A* expansion loop:

```
// In your A* expansion loop, after computing f_neighbor:
#ifdef DEBUG
    assert(f_neighbor >= f_current - 1);
    // Allow small tolerance (-1) for integer rounding if any;
    // ideally this should be f_neighbor >= f_current exactly.
#endif
```

Listing 6: Sanity check: f-values should be non-decreasing

If this assertion fires, check that the heuristic is being computed from the same cost table as the edge costs.

8.3 Expected cost ranges

With PAM250 and gap penalty 8, you should see:

Pair type	**Example**	**COST value**
Identical, highly conserved	Trp–Trp	−17
Identical, moderate	Ala–Ala	−2
Conservative substitution	Ile–Val	−4
Neutral substitution	Ala–Gly	−1
Radical substitution	Asp–Phe	+6
Gap	any–gap	+8

Note that individual COST values can be negative. Edge costs (sums of pairwise COST values) can also be negative. This is expected and handled correctly by A* as described in Section 2.3.

9 Benchmark Sequences

For benchmarking, use the same datasets as Lermen and Reinert (2000), which are publicly available and well-understood:

- **McClure globin dataset**: protein sequences from various globin families. The paper uses subsets of 5 and 11 sequences of varying lengths.
- **Small test case**: align 3–4 short sequences (length 20–50) first to verify correctness before moving to larger instances.

Sequences can be obtained in FASTA format from the NCBI protein database (`https://www.ncbi.nlm.nih.gov/protein/`) by searching for "globin" and filtering by organism.

10 Reference

- Lermen, M. and Reinert, K. (2000). The practical use of the A* algorithm for exact multiple sequence alignment. *Journal of Computational Biology*, 7(5):655–671.
- Dayhoff, M.O., Schwartz, R.M., and Orcutt, B.C. (1978). A model of evolutionary change in proteins. *Atlas of Protein Sequence and Structure*, 5(3):345–352.